

Torque

Put an AI agent to work on real Salesforce orgs — and keep it structurally unable to write to production on its own.

Omid Mojtahedi github.com/omoji-personal/torque MIT

Hand real Salesforce delivery work to an AI agent — deployments, data fixes, org audits, UI verification — and keep production structurally out of its reach.

Agents are now genuinely good at the mechanical half of Salesforce work. The reason most consultants still don't point one at a client environment is simple: the same session that fixes a validation rule holds credentials that can damage a live org, and "I told it to be careful" is not a control you can put in front of a client.

Torque is the control. It sits in front of every command the agent issues, authorises against the live org rather than a name, and makes a production write impossible for the agent to perform on its own. You get the speed on the work that should be delegated, and a boundary you can describe to a client without flinching.

WHO IT'S FOR	WHAT CHANGES
Consultants and admins running client orgs	Delegate the mechanical work without babysitting every command
Agencies deploying AI into customer environments	An answer to "what can it touch, and prove it" that is runnable, not asserted
Anyone using Claude Code against Salesforce	A safety layer that installs in one command and doesn't block ordinary work

What you can hand over

Each of these runs against a live org and is verified end to end — by checking the org afterwards, not by trusting a status code:

Deploy and verify	Push metadata to a sandbox, then confirm by SOQL that the field exists, that field-level security actually applied, and that nothing was left behind afterwards.
Mass-update safely	Preview the exact record set before touching it, apply the change, verify it persisted — and undo it back to the captured before-values.
Audit an org	Survey configuration, automations and permissions read-only, with no ability to change what it is inspecting.
Verify in the browser	Log in through a session handoff and confirm the page actually rendered for a user — because a green deploy is not a working screen.
Keep a record	Every change logged with before and after values, redacted of secrets, so a rollback is possible tomorrow and a client can be told what happened.
Leave it running	Work an org without watching every command, because the dangerous ones cannot execute without you personally approving them at your own terminal.

Why this is different

Most “AI guardrails” are instructions inside a prompt. They work until the model is three steps into a task and pattern-matching hard — precisely when you need them. Torque never asks. Two hooks sit in front of every tool call and *parse* the command rather than reading it, so an attempt like `x=sf; $x data delete bulk --target-org acme-prod` is refused even though the literal letters `sf` never appear.

The second difference matters more: **it is built to be checked, not believed**. Everything it claims is exercised against a real org by a harness you can run yourself, and the harness deliberately breaks its own safety checks to prove they still catch what they promise. There is a demo you can run in thirty seconds with no org and no credentials:

```

DENIED    x=sf; $x data delete bulk --target-org acme-prod
             a shell variable assembles the command at runtime

DENIED    cat ~/.torq*/secret
             a glob that only becomes the signing key's path once bash expands it

DENIED    git checkout HEAD~5 -- hooks/
             quietly restoring an older, weaker version of the guard itself

allowed   sf data query --query "SELECT Id FROM Account" -o any-org
             reads need no approval — ordinary work is untouched

```

That last line is the part most guardrails get wrong. **A guardrail that blocks real work gets switched off in week two.**

How it came to be

This didn't start as a product. It started as a problem I kept having.

THE SETUP	I spend my days delivering Salesforce for client organisations, increasingly with an AI agent doing the mechanical work. The speed is not subtle. Neither is the exposure: the same session that fixes a validation rule has the credentials to damage a live org.
THE REALISATION	I had written the careful-instruction prompts myself. Watching them hold most of the time — and knowing “most of the time” is not a safety property when the downside is a client's live data — was the moment it became an engineering problem instead of a policy one.
THE REBUILD	The practices that had accumulated across those engagements got rewritten from scratch as something general: no client specifics, no employer specifics, usable by anyone against any org.
THE ADVERSARIES	Rather than trust my own testing, I pointed independent adversarial reviewers at it whose only job was to defeat the guards — one of them executing each exploit against a real Salesforce CLI rather than reasoning about the code. They got through. Repeatedly.
WHAT THAT TAUGHT	They found a shell variable that assembles a command at runtime; a glob that only becomes a secret's path once the shell expands it; two redirects on one line where the parser saw only the second. Each one is now a permanent test. The uncomfortable one was in my own test suite: a check that verified field-level security, then returned success regardless of the answer. It would have reported green while the thing it was checking was missing entirely.

That last discovery shaped the whole posture. It is easy to build something that looks safe. The only defence against looking safe is to keep trying to break it, and to make every check prove it can still fail.

How it holds

Connect production read-only. The strongest protection is not granting the capability. Torque stores no credentials of its own.

Authorise by identity, not inference. A write is allowed only when its target is on an explicit list *and* a live query at that moment confirms it isn't production. An org that cannot be verified is treated as production.

Require a human for the dangerous things. Bulk deletes, unbounded updates and anonymous Apex need approval that is minted only at a real terminal, is single-use, and expires. The agent can ask for it and provably cannot create it.

Make production deliberate, not impossible. Real work includes deploying to a client's production org — so you type `WRITE PRODUCTION` yourself and get a token for one operation, or open a short revocable window. Every production write is recorded.

Where the boundary honestly sits

The guards cover what the agent does through its tools. They do not claim to stop someone with your login who steps outside those tools entirely — reading the signing key by other means, or having the agent write a script and run it so the CLI executes where no hook can see. Closing that last channel properly requires intercepting the CLI itself, which is the next version and is **not built yet**.

Which is why the first protection is the one that doesn't depend on any of this: connect production read-only, and never leave a production org authenticated in a session running on its own. A safety claim you cannot reproduce is marketing, not security — so this document says what is not covered, in the same place it says what is.

Try it

```
git clone https://github.com/omoji-personal/torque
python3 bin/torque-demo                # 30 seconds, no org, no credentials

sf org login web --alias my-dev-org
python3 bin/torque-init my-dev-org      # refuses to allowlist a production org
python3 harness/validate.py --profile release --target-org my-dev-org
```

A free Developer Edition org is enough. `torque-init` proves the guards actually bind before it tells you it worked, and refuses to configure a production org at all.

Found a way through? That is the point — every test in the suite exists because a review beat an earlier version of this code. `SECURITY.md` has the disclosure path. MIT licensed; use it, fork it, take the parts that help.